ALISSON GABRIEL DOS SANTOS ALVES

Inkdroid: Repurposing de tablets legados via debloat automatizado para leitura focada

Salvador

2026

ALISSON GABRIEL DOS SANTOS ALVES

Inkdroid: Repurposing de tablets legados via debloat automatizado para leitura focada

Trabalho de Conclusão de Curso apresentado ao Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, como parte dos requisitos para obtenção do Certificado de conclusão de curso.
Curso: Técnico em Informática

Orientador: Gilberto Monteiro

Salvador
Ano 2026

Agradecimentos

Agradeço em primeiro lugar ao meu orientador, Prof. Gilberto Monteiro, pela paciência, aconselhamentos e suporte que prestou a mim e aos meus colegas, e que permitiram que eu sempre tivesse uma direção e caminho para fazer um bom trabalho.

À minha mãe, Sra. Valdeci, por sempre me apoiar e por ser o alicerce fundamental em todas as etapas, incentivando meus estudos e sempre exigindo que eu desse meu melhor.

Aos meus colegas e amigos, pelo companheirismo nas disciplinas, pela troca de experiências e dicas que sempre me ajudaram, pelas discussões e opiniões e por me ouvirem quando queria treinar minhas apresentações e falas.

Ao meu ex-tutor e amigo Pedro Francisco, e a todos do Maria Felipa Lab, onde tive diversas experiências que me acompanharam e ajudaram durante a elaboração desse projeto.

E à minha querida melhor amiga Bianca, pelos últimos 5 anos em que sempre esteve comigo em todos os momentos bons e ruins, e sempre acreditou no meu potencial.

A todos que, direta ou indiretamente, colaboraram para a realização deste trabalho, manifesto minha sincera gratidão.

Resumo

ALVES, Alisson Gabriel dos Santos. **Inkdroid: Repurposing de tablets legados via debloat automatizado para leitura focada**. 2026. Trabalho de Conclusão de Curso (TCC) – Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, Salvador, 2026.

Analisa o impacto do padrão de consumo linear na indústria de *hardware mobile* e o fenômeno do envelhecimento de software (*software aging*), que gera descartes precoces de dispositivos funcionalmente íntegros. Define como objetivo central o desenvolvimento de uma solução de software para promover a preservação ambiental e a saúde digital por meio da ressignificação (*repurposing*) de tablets Android legados em dispositivos de leitura focada (*e-readers*). Utiliza como objeto de estudo o tablet Multilaser M8 4G da rede pública estadual da Bahia. Emprega uma metodologia aplicada e experimental, fundamentada no uso da ferramenta Android Debug Bridge (ADB) para intervenções não invasivas em três camadas sistêmicas: aplicação, *framework* e *kernel*. O método compreende a execução de um *debloat* radical para remoção de serviços redundantes, o ajuste de parâmetros ergonômicos como escala de cinza e densidade de pixels para mitigar a Síndrome da Visão do Computador (CVS), além da aplicação de *underclock* e redução da taxa de atualização da tela para otimização energética. Valida a eficácia da intervenção por meio de análise quantitativa de desempenho e autonomia, utilizando métricas de *Throughput*, *Slowdown* e o modelo de Desejabilidade Composta para equilibrar performance e consumo. Conclui que a engenharia de software consciente permite a conversão de potenciais resíduos eletrônicos em ativos educacionais valiosos, comprovando a viabilidade técnica de políticas de TI sustentáveis e a democratização do acesso a ferramentas de leitura fluida e livre de distrações.

Palavras-chave: Debloat Android. Green IT. Saúde Digital. Obsolescência de Software. Deep Work.

Abstract

ALVES, Alisson Gabriel dos Santos. **Inkdroid: Repurposing legacy tablets via automated debloat for focused reading**. 2026. Graduation Thesis (TCC) – Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, Salvador, 2026.

This research analyzes the impact of linear consumption patterns in the mobile hardware industry and the phenomenon of software aging, which triggers the premature disposal of functionally sound devices. The central objective is the development of a software solution that combines environmental preservation and digital health by repurposing legacy Android tablets into focused reading devices (E-readers). The study specifically focuses on the Multilaser M8 4G tablet distributed within the Bahia state public school system. The methodology is classified as applied and experimental, utilizing a mixed approach to validate hardware repurposing. The technical procedure employs the Android Debug Bridge (ADB) for non-invasive interventions across three systemic layers: application, framework, and kernel. The method involves automated debloating to remove redundant services and Google Mobile Services (GMS), the adjustment of ergonomic parameters such as grayscale and pixel density to mitigate Computer Vision Syndrome (CVS), and kernel-level underclocking to maximize energy efficiency. The effectiveness of the intervention is validated through quantitative performance and autonomy analysis, using Throughput and Slowdown metrics alongside Composite Desirability models to balance performance and power consumption. It concludes that conscious software engineering enables the conversion of potential electronic waste into valuable educational assets, proving the technical viability of sustainable IT policies and the democratization of distraction-free reading tools.

Keywords: Android Debloat. Green IT. Digital Health. Software Obsolescence. Deep Work.

Sumário

1 INTRODUÇÃO 7

1.1 Problema de Pesquisa 7

1.2 Objetivos 7

2 JUSTIFICATIVA 9

3 METODOLOGIA 10

4 DESENVOLVIMENTO E IMPLEMENTAÇÃO 12

4.1 Intervenções em Nível de Aplicação 12

4.1.1 Configuração do Ambiente e Protocolo de Comunicação 12

4.1.2 Lógica de Automação e Processamento de Bloatwares 13

4.2 Intervenção em Nível de Framework 15

4.2.1 Conceituação e Arquitetura de Framework 15

4.2.2 Fluxo de Execução e Pipeline 15

4.2.3 Estratégia de Modificação Não Invasiva 17

4.2.4 Display e Síndrome da Visão do Computador 17

4.3 Bateria, Autonomia e Foco 19

4.4 Lógica de Programação, Arquitetura e Ferramentas do InkDroid . . 20

4.4.1 Evolução do Script, Modularidade e Modos de Execução 20

4.4.2 Verificação de Segurança e Preparação do Ambiente 20

4.4.3 Inicialização de Logs, Leitura da Blacklist e Debloat Automático 20

4.4.4 Download, Instalação de Aplicações e Definição do Launcher Padrão 21

4.4.5 Otimizações Ergonômicas do Display via Código 21

4.4.6 Ajustes Sistêmicos de Bateria e Funcionalidades da CLI 22

4.4.7 Arquitetura de Reversibilidade (`InkDroid_uninstall.sh`) 22

4.4.8 O Utilitário `fetchbook` e Resiliência de Rede 23

4.4.8.1 Arquitetura Multicamada de Busca 23

4.4.8.2 Integração Híbrida Bash-Python 24

4.4.8.3 Transferência Automatizada para o Dispositivo (`pushToDevice`) 24

5 RESULTADOS E DISCUSSÃO 26

5.1 Consolidação da Primeira Versão Estável 26

5.2 Objetivos Alcançados 26

5.3 Limitações e Desafios de Intervenção em Nível de Kernel 28

6 CONSIDERAÇÕES FINAIS 30

REFERÊNCIAS BIBLIOGRÁFICAS 31

GLOSSÁRIO 32

APÊNDICE A – ACESSO AO REPOSITÓRIO DE CÓDIGO ABERTO 38

1 INTRODUÇÃO

A crise ambiental contemporânea é considerada por muitos o problema de maior relevância da atualidade, sendo esta muito impulsionada pelo atual padrão de consumo linear da indústria no geral. Esse padrão se mostra ainda mais forte no contexto de *hardware mobile*, que é renovado a cada ano, prometendo grandes mudanças, deixando em pouco tempo aparelhos funcionais obsoletos ou ”artificialmente incapazes”. Parnas (1994) diz que esse efeito está fundamentalmente ligado ao que ele chama de *software aging*, no qual, à medida que as aplicações se atualizam, elas exigem cada vez mais do hardware, ainda que o mesmo seja perfeitamente capaz, tornando o software um ”fardo oneroso”. Essa prática impulsiona o aumento da geração de resíduos sólidos, em especial o lixo eletroeletrônico, que possui metais pesados de difícil decomposição. Segundo o relatório *The Global E-waste Monitor* (BALDé et al., 2024), a geração de lixo eletrônico, estimada em 62 bilhões de kg no ano de 2022, cresce cinco vezes mais do que a reciclagem documentada, com apenas 22,3% dos resíduos coletados adequadamente em 2022.

1.1 Problema de Pesquisa

A ”incapacidade artificial” nos dispositivos móveis é impulsionada pelo *overhead* de processamento do *Google Mobile Services* (GMS), pela evolução das camadas de *runtime* (ART) e pelas atualizações do sistema que vêm perfeitamente otimizadas para o hardware de última geração, mas com grandes falhas de otimização para os mais antigos. Isso deixa dispositivos com hardware perfeitamente funcional, freados pelo software.

1.2 Objetivos

A presente pesquisa tem por objetivo o desenvolvimento de uma solução de software que combine preservação ambiental e saúde digital por meio do *repurposing* de tablets Android em *e-readers*, dando um novo significado funcional para dispositivos obsoletos. Em específico, o tablet Multilaser M8 4G distribuído pelas escolas da rede pública estadual da Bahia.

Nesse dispositivo, será realizado um *debloat* automatizado para otimizar o uso do armazenamento, memória RAM e ciclos de CPU, junto à implementação de uma interface monocromática e ajustes ergonômicos para redução de fadiga visual. Tal preocupação com a fadiga visual e a ergonomia vem do crescente índice de CVS (Síndrome da Visão do Computador), a qual atinge cerca de 70% da população mundial (Sá, 2016). Além disso, outra preocupação da presente pesquisa é a validação da autonomia energética,

conforme Monteiro et al. (2023), o perfil do usuário e serviços de fundo são os maiores vilões da bateria. A automatização também terá como meta o ajuste de parâmetros do *kernel* para maximizar a autonomia energética. Essa automação será disponibilizada de forma *open-source* em portais como o GitHub de forma gratuita e em linguagens como Shell Script (Bash) e Python.

2 JUSTIFICATIVA

A necessidade dessa pesquisa é fortemente fundamentada na urgência de combater o modelo de consumo linear consolidado atualmente. Em 2022, a categoria de resíduos sólidos, que inclui aparelhos eletrônicos como celulares, tablets e laptops, chegou a acumular 5 bilhões de kg de lixo (BALDé et al., 2024). Entretanto, o destino da maioria dos dispositivos obsoletos não é o descarte, mas sim a chamada hibernação doméstica. Estima-se que famílias da União Europeia possuam cerca de 74 itens eletrônicos em suas casas, dos quais boa parcela está quebrada ou sem uso. No Brasil, a Associação Brasileira de Reciclagem de Eletroeletrônicos e Eletrodomésticos (ABREE, 2024) também sinaliza que cerca de 85% dos brasileiros possuem algum tipo de aparelho eletrônico parado em suas casas. O Brasil também é o maior gerador de *e-waste* da América Latina segundo a ABREE (ABREE, 2024). Esses dados representam um panorama preocupante em que, por diversos fatores, dispositivos legados em seu pleno funcionamento são encontrados no seu estado de hibernação. A presente pesquisa pode ajudar a mitigar os danos dessa situação, dando um novo propósito para aqueles dispositivos que já não servem bem às exigências dos softwares e aplicativos modernos.

Somado a isso, o desenvolvimento da pesquisa também ancora-se nos conceitos de saúde digital e minimalismo para promover o *Deep Work* por meio de modificações em várias das partes que compõem o sistema Android. De acordo com Newport (2019), a tecnologia dos dias de hoje é projetada para capturar nossa atenção, o que, em dispositivos de múltiplas funções, mostra-se um agente de fragmentação do foco, com diversos serviços disputando o seu olhar e atrapalhando o chamado *Deep Work*. A proposta de ressignificação visa também combater essa natureza de modo a tornar o aparelho um dispositivo de leitura livre de distrações.

Finalmente, o projeto possui um forte componente de conveniência e democratização tecnológica. Ao transformar um dispositivo comum, como o Multilaser M8 4G da rede estadual, em uma ferramenta educacional fluida, o estudo remove a barreira financeira imposta pelo alto custo de dispositivos especializados como o Kindle. Trata-se de uma prova de conceito de que a Engenharia Verde pode converter um dispositivo que seria trocado e descartado em um ativo educacional valioso e acessível.

3 METODOLOGIA

A presente pesquisa classifica-se como aplicada e experimental, com abordagem mista para validar a eficácia da ressignificação do hardware. O procedimento técnico central vai utilizar o Android Debug Bridge (ADB) como principal ferramenta, permitindo a comunicação entre a automação e o tablet de forma nativa e segura. Trata-se de uma ferramenta de CLI (*Command Line Interface*) oficial de desenvolvimento disponibilizada pelo *Android Open Source Project* (AOSP). Operando no modelo cliente-servidor, ela dá acesso ao Shell Unix dos dispositivos Android de forma não invasiva e sem necessidade de acesso root (ANDROID DEVELOPERS, 2025). Essa ferramenta possibilita a escrita de um código para realizar alterações profundas nos dispositivos, alterações que ocorrerão em três camadas principais:

1. **Nível de Aplicação**: Nesta etapa o script vai, de forma automática, instalar os aplicativos necessários para o pleno funcionamento do dispositivo como um *e-reader*, e remover processos redundantes ou aplicativos pré-instalados (*bloatwares*), fazendo um *debloat* completo. Desse modo, além de remover redes sociais, apps que prejudicam o *deep work* e atrapalham o foco, também serão removidos, caso o usuário opte, os chamados *Google Mobile Services* (GMS) que incluem telemetrias intrusivas. Esta ação libera recursos desperdiçados em coisas desnecessárias e constrói a base para as modificações mais profundas.

2. **Nível de Framework**: Essa é a fase em que ocorre a manipulação do banco de dados `settings.db` e do gestor de janelas (*Window Manager* ou WM) para alterar diversas das configurações padrão. Nessa fase, algumas das principais modificações vão ser a aplicação de um filtro de luz azul nativo, simulando o *e-ink* de *e-readers* especializados. Essa alteração mostra-se relevante pois, segundo Benedetto et al. (2013), as telas *e-ink* diminuem drasticamente o risco de CVS, e embora a tela continue sendo LCD, a redução da luz azul e a escala de cinza reduzem o brilho e o contraste agressivo, mitigando a CVS; e, embora não atinja as propriedades físicas de reflexão de luz do papel eletrônico real, aumentará o conforto ocular do usuário. A manipulação da densidade de pixels (DPI) e da escala de cinza também será aplicada visando otimizar a renderização de fontes, reduzindo a carga cognitiva e visual durante a leitura (Sá, 2016; ANDROID DEVELOPERS, 2025).

3. **Nível de Kernel**: Por fim, essa é a parte de maior dificuldade, mas que vai aplicar as maiores diferenças práticas. Aqui, a automação vai aplicar técnicas de *underclock* (limitação da frequência do processador) e redução da taxa de atualização da tela

via *SurfaceFlinger*, para tornar a tela do aparelho mais semelhante ao padrão da indústria, além de ajudar com a eficiência energética.

Essas modificações serão acompanhadas de uma análise de dados para medir a diferença de desempenho, uso de recursos de hardware e autonomia da bateria, por meio de uma comparação quantitativa usando métricas como *Throughput* e *Slowdown* para comprovar se a alteração foi de fato benéfica ou não do ponto de vista de fluidez do sistema, utilizando modelos de Desejabilidade Composta para equilibrar desempenho e consumo energético (PONTES, 2017).

4 DESENVOLVIMENTO E IMPLEMENTAÇÃO

O presente capítulo descreve os procedimentos técnicos realizados para a elaboração da solução, estruturando-se conforme os níveis sistêmicos estabelecidos na metodologia: Aplicação, Framework e Kernel. A implementação foi conduzida em ambiente desktop Linux, visando integração nativa com ferramentas de automação via terminal.

4.1 Intervenções em Nível de Aplicação

O primeiro nível de intervenção foca na desoneração do sistema através da remoção de processos e serviços irrelevantes para o produto final, bem como a integração das ferramentas necessárias para estabelecer uma experiência de leitura focada. Entretanto, antes de iniciar-se a etapa de desenvolvimento prático, faz-se necessária a preparação do ambiente e das ferramentas que vão permitir a comunicação entre os dispositivos.

4.1.1 Configuração do Ambiente e Protocolo de Comunicação

O desenvolvimento se inicia pela instalação e preparação da ferramenta *Android Debug Bridge* (ADB), que, em consonância com a metodologia estabelecida, trata-se de uma ferramenta de linha de comando (*Command Line Interface* – CLI) oficial do AOSP (ANDROID DEVELOPERS, 2025), que será responsável por cada um dos comandos e modificações realizadas ao longo do processo. A instalação do ADB, em especial em ambientes Linux desktop, caracteriza-se pela simplicidade, se resumindo a um único comando do gerenciador de pacotes da distribuição. Contudo, apesar do rápido providenciamento, a ferramenta possui uma arquitetura robusta e abrangente.

Em termos funcionais, o ADB opera em uma arquitetura de cliente-servidor (TANENBAUM; BOS, 2016), a qual permite a troca de informações e o envio de instruções entre os dois dispositivos; nessa estrutura, o servidor (executado no tablet) atua de forma reativa, processando requisições e retornando dados, enquanto o cliente (executado na estação de trabalho) detém a prerrogativa de ordenar as execuções. Paralelamente, estabeleceu-se o controle de versão via repositório Git, permitindo o rastreamento de modificações e a organização colaborativa do código-fonte, parte crucial do projeto visto que o mesmo tem por objetivo se manter *open-source*[1]. A manutenção do código em formato aberto

[1] Open Source é uma forma de manter projetos de software na qual o autor disponibiliza todo seu código (*source*) publicamente para que outras pessoas ou empresas possam usar ou contribuir, desde que sigam as regras da sua licença.

(*open-source*) justifica-se pela necessidade de auditabilidade e transparência, sobretudo em um projeto que realiza modificações profundas no sistema operacional.

No entanto, por conceder acesso ao *shell Unix*[2] do celular, a abertura desse protocolo pode se tornar um vetor de invasão preocupante sem as devidas proteções. Para mitigar tais riscos, o sistema Android implementa medidas de segurança rigorosas fundamentadas em permissões de usuário, *sandboxes* e na autenticação por chaves de criptografia assimétrica RSA. No ecossistema Android, cada um dos aplicativos roda em um ambiente isolado e tem um identificador de usuário único e exclusivo (UID) com permissões diferentes. Isso garante que, por padrão, nenhum programa tenha acesso a dados dos outros, nem ao sistema. Embora o Android, diferente de outros sistemas, não disponibilize um usuário root (administrador) por padrão, os comandos do ADB são executados por um usuário especial com permissões elevadas chamado `shell`. Esse usuário pode gerenciar permissões de outros, ler *logs*[3] de sistema, alterar configurações internas e desinstalar e instalar programas.

Para mitigar acessos não autorizados a esse usuário privilegiado, o protocolo utiliza a criptografia assimétrica. Até a versão 4.2.2 do Android, bastava ativar a depuração e qualquer máquina poderia executar comandos no *shell*, criando uma vulnerabilidade como a adulteração de aparelhos furtados ou ataques por meio de estações de carregamento maliciosas (*juice jacking*) em locais públicos. Para corrigir isso, o Google implementou uma autenticação por chave RSA, que funciona com uma chave pública (`adbkey.pub`) e uma privada (`adbkey`). Quando se conecta um computador, o dispositivo solicita autorização e, ao selecionar "Sempre permitir a partir deste computador", uma cópia da chave pública é salva no celular para validar as conexões futuras e garantir a integridade da automação sem violar os protocolos de segurança nativos do sistema.

4.1.2 Lógica de Automação e Processamento de Bloatwares

Consolidados o ambiente de desenvolvimento e o repositório Git, o desenvolvimento da automação inicia-se através de uma verificação inteligente dos aparelhos servidores conectados. Essa medida visa assegurar que o *script* execute apenas quando houver exatamente um tablet conectado ao barramento USB, evitando possíveis conflitos entre os comandos, assim como previne que a automação rode sem nenhum dispositivo conectado.

Validada a conexão, a próxima fase seria a desoneração do sistema via remoção de softwares prescindíveis (*debloat*). A desinstalação de pacotes do dispositivo não se restringe a uma otimização superficial, mas constitui uma estratégia para conter o envelhecimento de software (*software aging*). Conforme fundamentado por Parnas (PARNAS, 1994), o envelhecimento de software ocorre quando o programa se torna um "fardo oneroso" devido

[2] Unix foi um dos primeiros e mais importantes sistemas operacionais, estabelecendo padrões usados até hoje inclusive no Android, Linux e macOS.

[3] Logs são registros do que acontece em tempo de execução de uma aplicação, usados para monitoramento e diagnósticos.

ao acúmulo de atualizações e serviços contínuos, esses que são voltados para o hardware mais novo, mas também são empurrados para o dispositivo legado, demandando ainda mais recursos computacionais. Ao eliminar o excesso de processos, a automação restaura a fluidez do aparelho e diminui os agentes de distração, permitindo que a execução de tarefas simples como a leitura seja mais confortável e, principalmente, conveniente.

Para estruturar o *debloat* sem poluir o código principal com chamadas maçantes, foi adotada uma abordagem declarativa baseada no arquivo de configuração `blacklist.json`. Esse arquivo lista os nomes exatos de pacotes de redes sociais, ferramentas do *Google Mobile Services* e demais *bloatwares* identificados como prejudiciais ao foco ou ao desempenho do sistema. A escolha da remoção do GMS fundamenta-se na eficiência energética: estudos de mapeamento sistemático indicam que aplicativos e serviços que operam em segundo plano são os principais responsáveis pela drenagem de energia em dispositivos Android (MONTEIRO et al., 2023). A eliminação desse tipo de tarefa previne que o dispositivo acorde da hibernação por motivos irrelevantes, maximizando a autonomia em *idle*.

O formato JSON confere flexibilidade ao projeto por sua sintaxe de fácil leitura e modificação, permitindo ao usuário adicionar ou remover *apps* da lista se necessário com facilidade. Sob a perspectiva conceitual, essa modularidade alinha-se aos preceitos do Minimalismo Digital (NEWPORT, 2019), o qual preconiza que a tecnologia deve ser configurada de forma seletiva e intencional.

Porém, para o script lê-lo e manipulá-lo de forma efetiva, foi necessária a integração da ferramenta de linha de comando `jq`, implementada em uma estrutura de repetição que percorre toda a lista e executa uma tentativa de desinstalação para cada objeto. Em adição, o laço de repetição também salva um arquivo de *log* com o resultado de cada um dos comandos, o que permite a auditoria do processo e validação da execução. A intenção é que todas as camadas salvem seus resultados no arquivo de *log*.

Ao término do *debloat*, a camada de aplicação já está quase completa, restando agora a instalação das ferramentas que serão essenciais no uso do produto final. Instalar ferramentas no Android usando o ADB está longe de ser uma tarefa complicada; o verdadeiro desafio é saber de onde baixar os pacotes (APKs), já que a Google Play Store, por exemplo, não expõe nenhum tipo de API. Para contornar esse desafio, optou-se por utilizar aplicativos *open-source* disponíveis diretamente dos *releases* do GitHub, o qual possui uma API acessível. Do âmbito da ergonomia cognitiva do usuário, trocar o launcher padrão significa remover elementos gráficos apelativos e notificações projetadas para reter atenção.

4.2 Intervenção em Nível de Framework

Nesta seção, exploram-se o funcionamento dos *frameworks* e os processos de gerenciamento do sistema Android, bem como as modificações que o InkDroid fará no tablet-alvo nessas camadas. Entretanto, é importante deixar claro que a denominada camada de *framework* nesta pesquisa é, na verdade, um grupo de outras quatro camadas menores compostas por: APIs de sistema, Android Runtime (ART), *daemons* de sistema, bibliotecas nativas (em C ou C++) e a HAL.

4.2.1 Conceituação e Arquitetura de Framework

Segundo (TANENBAUM; BOS, 2016), sistemas operacionais foram criados tendo como principal objetivo serem uma forma de abstração para o desenvolvimento de software, fazendo com que a partir de sua criação, o desenvolvedor de programas comuns não precisasse mais se preocupar com a complexidade do hardware, permitindo desenvolver sem ter um entendimento avançado de eletrônica ou saber, por exemplo, como gerenciar a voltagem de um chip de memória. Quando nos referimos aos *frameworks* do sistema, estamos falando sobre várias dessas abstrações organizadas que formam uma pilha de software (*software stack*) contendo APIs, bibliotecas, SDKs, *daemons* etc., além de outras peculiaridades diferentes para cada sistema.

4.2.2 Fluxo de Execução e Pipeline

Para entendermos como se dá o funcionamento dessas abstrações do sistema Android, vamos usar um exemplo simples. Imagine que um desenvolvedor quer que seu aplicativo reproduza um efeito sonoro simples; se ele fizesse isso sem utilizar nenhuma abstração, precisaria lidar com diferenças entre vários dispositivos de som diretamente em seu código, mas usando a API do Android, tudo o que precisa fazer é solicitar a reprodução do arquivo de áudio ao sistema operacional. Essa instrução então passa por todas as camadas até chegar finalmente ao hardware e reproduzir o som de fato.

O primeiro lugar por onde essa instrução passa é na Java API; aqui é onde o sistema vai validar várias informações antes de iniciar o processo pesado, fazendo perguntas como: "todos os serviços do sistema de som foram iniciados corretamente?". Depois, essa requisição passa pela JNI (*Java Native Interface*) que vai instanciar uma classe do cliente de áudio (`audioserver`), que é o *daemon* responsável por gerenciar serviços de áudio (ANDROID DEVELOPERS, 2025; AOSP, 2024).

O conceito de "classe" não vem ao caso agora, mas nessa situação, você pode pensar nela como uma abstração para tarefas que o `audioserver` vai executar. Através dela, componentes que rodam em Java ou Kotlin podem interagir com esse processo, que roda em C++ (linguagem usada na maioria dos *daemons* do Android); sendo assim, é como

se cada tarefa do *daemon* com a qual componentes do ART ou JVM precisam interagir tivesse sua representação em uma classe.

Agora a situação é: o *daemon* já sabe da intenção do aplicativo de reproduzir um som, e a responsabilidade dele é preparar tudo o que for necessário para reproduzi-lo. Uma série de outros processos mais específicos e técnicos acontecem nesse meio tempo, então vamos focar apenas nos que são relevantes para a natureza do InkDroid. Dentre eles, o primeiro é a validação de estado do aparelho.

O *daemon* é um serviço de sistema e, como o padrão de segurança do Android é isolar os processos em *sandbox*, ele roda em uma parte do sistema chamada *Daemon Space* que não consegue acessar diretamente o chamado *User Space*, onde estão as informações de estado. Por conta desse isolamento, o `audioserver` precisa fazer uma requisição IPC via Binder, o barramento nativo de IPC do Android, que empacota e envia a mensagem para o AudioService, módulo de áudio que roda no *User Space* (AOSP, 2024).

O AudioService é o componente que controla configurações como o volume definido pelo usuário, o modo silencioso, qual aplicativo detém o "*Audio Focus*", quem possui permissão para reproduzir áudio e outros parâmetros. O `audioserver` recebe a resposta e imediatamente a submete às suas próprias políticas e regras, que comparam as informações do *User Space* com o estado dos conectores físicos e determinam a rota que os dados vão fazer até o alto-falante.

Depois dessa validação, vem a preparação do dado. No exemplo atual, isso significa preparar o arquivo MP3, que usa técnicas de compressão de dados e por isso não pode ser reproduzido diretamente, necessitando de uma decodificação. Isso é feito através do motor de mídia Stagefright/Codec2, transformando o arquivo MP3 em um PCM (*Pulse Code Modulation*), formato que armazena a posição da membrana do alto-falante para cada milissegundo do áudio. É o movimento dessa membrana que cria as ondas sonoras.

Entretanto, o PCM gerado ainda não é aquilo que vai ser reproduzido: o *daemon* ainda utiliza o AudioFlinger para fazer a mixagem, isto é, consolidar todos os sons reproduzidos na mesma janela temporal em um único PCM.

O PCM resultante desce mais um nível até a HAL, mais especificamente no módulo de áudio da HAL fornecido pelo fabricante do SoC. Ele traduz todas as instruções das camadas mais altas em instruções específicas para o hardware, nesse caso, o chip de áudio do SoC. Ele configura a rota fornecida pelo `audioserver` para a saída de áudio e executa *system calls* para o *driver* de áudio do Linux, o ALSA (*Advanced Linux Sound Architecture*), comunicando-se com o DAC (*Digital to Analog Converter*), que converte o sinal digital do PCM (zeros e uns) em sinal analógico (níveis variáveis de energia). Esses sinais são enviados para o alto-falante, que finalmente reproduz o som quase instantaneamente (TANENBAUM; BOS, 2016; ANDROID DEVELOPERS, 2025).

O objetivo desse exemplo é entender o papel de cada um dos componentes de software citados e como eles se integram entre si, além de saber que para cada subsistema da plataforma existe um pipeline análogo, como na renderização gráfica, conectividade de rede, Bluetooth ou telefonia celular.

4.2.3 Estratégia de Modificação Não Invasiva

Com base na explicação da seção anterior e nas observações que Yaghmour (2013) faz em sua obra, nota-se que a camada de *framework* é extremamente complexa, delicada e interconectada, sendo peça central do sistema operacional; desse modo, modificar bibliotecas e executáveis compilados pode induzir inúmeros *softbricks*. Por conta disso, o projeto adotou uma abordagem segura para alterar o comportamento dos componentes da camada de *framework* sem recorrer à engenharia reversa de baixo nível: modificam-se diretamente as regras de negócio e políticas que regem os serviços do Android por meio de parâmetros de configuração nativos mantidos pelo sistema.

Segundo a documentação oficial (ANDROID DEVELOPERS, 2025; AOSP, 2024), o sistema Android armazena suas políticas e configurações em um banco de dados relacional baseado em SQLite, o qual gerencia as variáveis de comportamento da camada de *framework* e do *User Space*. O ADB dispõe de comandos específicos para consultar e alterar essas variáveis, respectivamente: `adb shell settings get` e `adb shell settings put`. Esse mesmo mecanismo é utilizado para definir o *launcher* padrão via linha de comando, alterando o identificador do pacote encarregado de responder à ação do botão *Home*.

Além do `settings.db`, subsistemas centrais como o Window Manager e o *SurfaceFlinger* possuem políticas e interfaces próprias para parametrização em tempo de execução.

4.2.4 Display e Síndrome da Visão do Computador

Sendo a tela o elemento central de uso e interesse em *e-readers* dedicados, ela constitui um pilar fundamental de intervenção nesta pesquisa. Para tanto, cumpre compreender os fatores ergonômicos da tecnologia *e-ink* e as limitações dos painéis LCD convencionais. Concebida originalmente para oferecer baixo consumo energético e refletividade semelhante ao papel, a tecnologia *e-ink* demonstrou posteriormente benefícios ergonômicos significativos na mitigação da CVS. A Associação Americana de Optometria (AOA) define a Síndrome da Visão do Computador como o conjunto de problemas oculares decorrentes do uso prolongado de telas digitais, cuja prevalência atinge cerca de 70% dos usuários em escala global (Sá, 2016).

No estudo de Benedetto et al. (2013), comprova-se experimentalmente que a taxa de piscamento por minuto é substancialmente menor durante o uso de telas LCD em

comparação ao papel ou ao *e-ink*. Sá (2016) quantifica esse comportamento: em telas LCD, a frequência de piscamento cai para 5 a 7 vezes por minuto em atividades de leitura concentrada, enquanto na leitura em papel atinge média de 22 vezes por minuto. Essa redução acelera a evaporação do filme lacrimal, desencadeando sintomas de ressecamento e fadiga ocular.

Adicionalmente, a emissão contínua de luz azul em telas retroiluminadas afeta os ritmos circadianos. Conforme destaca Knez (2001), a temperatura de cor e a intensidade luminosa influenciam diretamente processos psicológicos não visuais; a exposição noturna desregulada à luz azul inibe a síntese de melatonina, prejudicando a qualidade do sono e elevando a carga de estresse biológico.

Painéis *e-ink* minimizam esses efeitos por operarem de forma puramente reflexiva, aproveitando a luz ambiente e apresentando taxas de atualização estáticas que reduzem o esforço de acomodação visual. Embora um painel LCD convencional não possa alterar sua constituição física, Benedetto et al. (2013) demonstra que a fadiga visual em telas LCD pode ser significativamente atenuada via software por meio do controle de brilho, ampliação do contraste tipográfico e supressão da saturação de cores. Com base nessas premissas, o InkDroid implementa intervenções ergonômicas estruturadas no `settings.db`:

1. **Movimentos e Animações:** Um dos fatores primordiais para a ergonomia do *e-ink* reside na ausência de transições dinâmicas contínuas. No sistema Android, as animações de interface aumentam o *overhead* computacional e geram distrações visuais. Por meio das chaves documentadas pelo AOSP (ANDROID DEVELOPERS, 2025), a automação suprime todas as animações globais atribuindo valor 0 a `window_animation_scale`, `transition_animation_scale` e `animator_duration_scale` na tabela ”global”. Além disso, a taxa de atualização gráfica do painel é fixada em 30 Hz via *SurfaceFlinger*, reduzindo o consumo de energia da GPU e estabilizando a exibição.

2. **Cores e Conforto Visual:** Com a supressão de animações, a mitigação visual concentra-se na temperatura e saturação luminosa. O InkDroid oferece dois perfis de exibição: o perfil monocromático, que ativa a matriz de acessibilidade daltonizadora em escala de cinza para neutralizar estímulos cromáticos; e o perfil de filtro de luz azul, que atenua os comprimentos de onda de alta energia. Em paralelo, realizam-se ajustes de densidade de pixels (DPI) na tabela ”secure” para otimizar o conforto na leitura de fontes tipográficas.

Essas configurações transformam a experiência de leitura no tablet, proporcionando conforto visual substancialmente superior ao padrão de fábrica, além de minimizar travamentos e reduzir o consumo de bateria.

4.3 Bateria, Autonomia e Foco

A longevidade da carga e a ausência de interrupções constituem os principais atributos valorizados em um dispositivo de leitura dedicado. Enquanto aparelhos com telas *e-ink* atingem semanas de autonomia devido à tecnologia de exibição biestável, tablets Android comerciais mantêm dezenas de serviços concorrentes em segundo plano que esgotam a bateria e disputam ativamente a atenção do usuário.

Para conter esse desperdício de ciclos, o InkDroid ativa a diretiva `always_finish_activities`, impedindo a persistência desnecessária de rotinas em memória RAM. As principais diretivas de conservação energética e foco são estruturadas a seguir:

1. **Modo de Economia Permanente:** Diante da ausência de rotinas nativas de *underclock* sem privilégios de root, a automação eleva o patamar de ativação automática do modo econômico de 15% para 100% através do parâmetro `low_power_trigger_level`. Esse ajuste assegura que o perfil de máxima eficiência permaneça ativo de maneira contínua, mesmo após reinicializações.

2. **Conectividade Sem Fio Controlada:** O sistema operacional executa varreduras frequentes de rede mesmo quando desconectado. Ao definir as variáveis `wifi_scan_always_enabled` e `ble_scan_always_enabled` como 0, essas varreduras passam a operar estritamente sob demanda do usuário. Desativa-se igualmente a sincronização automática em segundo plano (`auto_sync`) e disponibiliza-se a opção de desabilitar os dados móveis (`mobile_data`) em dispositivos dedicados exclusivamente ao estudo.

3. **Sons, Notificações e Vibrações Hápticas:** O acionamento de atuadores hápticos e sonoros gera drenagem energética contínua e quebra o estado de foco. O script desativa a vibração em `haptic_feedback_enabled`, zera a intensidade em `haptic_feedback_intensity`, suprime notificações flutuantes (`heads_up_notifications_enab` e ativa o modo `zen_mode` (Não Perturbe).

4. **Configurações Adicionais de Estabilidade:** O tempo limite de desligamento da tela (`screen_off_timeout`) é ampliado para 1.800.000 milissegundos (30 minutos), prevenindo apagamentos acidentais durante a leitura concentrada de obras densas. Desativam-se também os *daemons* de escuta contínua de assistentes de voz e as atualizações automáticas OTA (*Over-The-Air*), resguardando a integridade das modificações efetuadas.

Essas intervenções sobre a camada de *framework* consolidam um ambiente minimalista, energeticamente eficiente e ergonomicamente adaptado para leitura prolongada.

4.4 Lógica de Programação, Arquitetura e Ferramentas do InkDroid

Tendo analisado a arquitetura do sistema e as alterações no nível de *framework* na seção anterior, detalham-se aqui a estrutura lógica do código-fonte, as rotinas de automação em *shell script* e o funcionamento das principais ferramentas desenvolvidas na CLI do InkDroid.

4.4.1 Evolução do Script, Modularidade e Modos de Execução

O desenvolvimento da ferramenta de instalação do InkDroid evoluiu para uma arquitetura modular orientada a funções em Bash, o que facilita o reaproveitamento de rotinas, a manutenção e a extensibilidade do código. A automação suporta execução interativa e não interativa por meio de interpretação de argumentos de linha de comando (*CLI flags*). Através da função `parseArguments()`, processam-se etapas específicas (como `-only-debloat`, `-only-apps` ou `-only-display`), a omissão de dependências (`-skip-deps`) e a confirmação automática (`-y` ou `-yes`). O controle do fluxo de execução é orquestrado pela função `main()`, que invoca condicionalmente cada sub-rotina de acordo com os parâmetros fornecidos.

4.4.2 Verificação de Segurança e Preparação do Ambiente

O fluxo de execução inicia-se pela função `installDependencies()`, que identifica o gerenciador de pacotes do sistema hospedeiro Linux (`apt-get`, `dnf`, `pacman` ou `zypper`) e garante a presença de ferramentas essenciais de manipulação, como `android-tools`, `jq`, `curl` e `bc`, assegurando que todos os componentes necessários estejam disponíveis e prevenindo interrupções durante o processo. Em seguida, a função `checkDevice()` consulta os dispositivos via comunicação ADB e contabiliza os aparelhos detectados em modo de depuração USB, interrompendo a execução caso nenhum ou múltiplos dispositivos sejam encontrados simultaneamente. Quando identificado um único aparelho compatível, a função extrai propriedades sistêmicas (`ro.product.manufacturer`, `ro.product.model`, `ro.build.version.release`) e invoca a rotina `confirmDevice()` para solicitar a validação de segurança pelo usuário, garantindo que o operador confirme a intervenção no dispositivo desejado.

4.4.3 Inicialização de Logs, Leitura da Blacklist e Debloat Automático

Validada a conexão, a rotina `initLog()` verifica a configuração no arquivo `blacklist.json` e cria o documento de auditoria `debloatlog.txt`, registrando no cabeçalho o modelo do dispositivo e o carimbo de data/hora da execução. O fluxo de desinstalação apoia-se em duas funções sequenciais. A primeira, `parseBlacklist()`, extrai os identificadores de pacotes utilizando `jq` ou um interpretador Python 3 como segunda opção (*fallback*). A

segunda, `debloatPackages()`, processa essa lista em um ciclo de repetição, executando a seguinte instrução: `adb shell pm uninstall -k -user 0 "$pacote"`. Esta lógica garante que os aplicativos sejam desinstalados exclusivamente no espaço do usuário principal, processando e registrando em *log* os retornos do sistema (`Success`, `not installed` ou falhas) em tempo real.

4.4.4 Download, Instalação de Aplicações e Definição do Launcher Padrão

O processo de instalação de dependências externas apoia-se primariamente na função `getReleaseApkUrl()`, que realiza requisições à API REST do GitHub via `curl` e processa os metadados JSON para extrair dinamicamente a URL do ativo (*asset*) APK mais recente. As funções de implantação gerenciam o pacote da seguinte forma: os metadados são interpretados para extrair a versão mais recente das aplicações (KOReader e Olauncher), realizando o download para `/tmp/KOReader.apk`, onde o arquivo é mantido temporariamente até sua instalação via `adb install -r -user 0`.

Para isolar o ambiente, a função `setDefaultLauncher()` aplica o comando `adb shell cmd role add-role-holder android.app.role.HOME app.olauncher`, forçando a substituição da interface padrão via *RoleManager* do Android. Esse passo restringe notificações flutuantes e estímulos gráficos, baseando-se nos preceitos do Minimalismo Digital (NEWPORT, 2019).

4.4.5 Otimizações Ergonômicas do Display via Código

Para implementar os perfis de mitigação visual estabelecidos anteriormente, o script utiliza a função `configureDisplay()`, que automatiza as injeções de parâmetros no banco de dados `settings.db`. A supressão das animações globais do sistema é realizada forçando os valores `window_animation_scale`, `transition_animation_scale` e `animator_duration_scale` para zero. Em seguida, o algoritmo processa a saída do serviço nativo via `dumpsys display` para isolar a menor taxa de atualização suportada pelo *hardware*, injetando limitadores correspondentes nas chaves `peak_refresh_rate` e `min_refresh_rate`. A execução dos perfis ocorre por meio dos seguintes ajustes:

- **Perfil Monocromático:** Manipula variáveis de acessibilidade injetando `accessibility_displa` `1` e definindo o índice de escala de cor com `accessibility_display_daltonizer` `0`.
- **Filtro de Luz Azul:** Altera a emissão do painel configurando `night_display_activated` `1` e estabelecendo a temperatura de cor em `night_display_color_temperature` `3000`.

4.4.6 Ajustes Sistêmicos de Bateria e Funcionalidades da CLI

A implementação das políticas de economia energética e restrição de conexões é orquestrada pela função `apply_Framework_Tweaks()`. A sub-rotina processa a injeção em lote diretamente nas tabelas `global`, `secure` e `system` do `settings.db`. É este módulo do código que ativa o encerramento de atividades em segundo plano (`always_finish_activities 1`), eleva permanentemente o limitador de economia (`low_power_trigger_level 100`), estende o tempo de bloqueio de tela para trinta minutos (`screen_off_timeout 1800000`) e desativa chaves de rastreamento contínuo (`wifi_scan_always_enabled`, `haptic_feedback_enabled`, `zen_mode`).

O projeto disponibiliza adicionalmente a interface de linha de comando `ink`, projetada para operar como um utilitário CLI interativo global com os seguintes subcomandos estendidos:

- `ink status`: Aciona a função `cmdStatus()` para consultar dinamicamente propriedades sistêmicas via `getprop`, capturar a bateria via `dumpsys battery`, e extrair a densidade e resolução de tela com `wm density` e `wm size`.
- `ink push <arquivo>`: Executa a rotina `cmdPush()` para transferir e-books (.epub ou .pdf) para o diretório `/sdcard/Books/` via `adb push`. Após a transferência, o script emite programaticamente um sinal (*broadcast*) ao `MEDIA_SCANNER` do Android, forçando a atualização indexada do KOReader.
- `ink display <profile>`: Invoca a função `cmdDisplay()`, processando argumentos textuais no terminal para aplicar, em tempo real, os modos `monochrome`, `blue` ou realizar o `reset` da exibição.

4.4.7 Arquitetura de Reversibilidade (`InkDroid_uninstall.sh`)

Para conformidade com os princípios de governança, o script de desinstalação reverte o estado do sistema ao padrão de fábrica do ecossistema do dispositivo, sem necessariamente formatá-lo, o que acarretaria a perda de arquivos pessoais. O processo lê os mesmos arquivos de configuração JSON para reabilitar os pacotes previamente desativados via:

```
adb shell cmd package install-existing <package_name>
```

Restaura também os parâmetros de densidade de tela originais e remove as aplicações e modificações introduzidas pelo projeto.

4.4.8 O Utilitário `fetchbook` e Resiliência de Rede

O utilitário `fetchbook` atua como a camada de aquisição de conteúdo do InkDroid. Projetado originalmente como um cliente de busca para repositórios abertos, o script passou por uma reformulação arquitetural profunda devido a transformações na infraestrutura da internet. A primeira versão do utilitário enfrentava degradação operacional severa causada pela instabilidade e apreensão de domínios públicos de bibliotecas abertas (*domain seizures*), além de bloqueios de borda com desafios JS/CAPTCHA, inviabilizando a raspagem de dados (*scraping*) via requisições diretas por `curl`.

A Tabela 1 resume a transição de arquitetura implementada na ferramenta para garantir alta disponibilidade:

Tabela 1 – Comparativo Arquitetural entre a Versão Legada e a Nova Abordagem Multicamada

Vetor de Análise	**Arquitetura Legada**	**Nova Arquitetura Multicamada**
Resolução de Fontes	Endpoints HTTP fixos / Espelhos únicos	Provedores encadeados com fallback automático
Parsing de Dados	Expressões regulares (`grep`/`sed`) em HTML	Parsing JSON nativo e estruturado via Python
Distribuição de Conteúdo	Download HTTP direto com dependência de host	Redes descentralizadas (Gateways IPFS)
Tratamento de Desafios	Falha genérica em bloqueios de borda	Redirecionamento a APIs públicas e protocolos P2P

4.4.8.1 Arquitetura Multicamada de Busca

Para assegurar o provimento contínuo de dados sem interrupções, o novo `fetchbook` estrutura a busca em três níveis hierárquicos de alta resiliência:

1. **Camada Primária (Project Gutenberg API):** Prioriza obras em domínio público com metadados estruturados e garantias de estabilidade de servidor.
2. **Camada Secundária (Open Library):** Atua como ponte para catalogação internacional e identificação de metadados via ISBN/OLID.
3. **Camada Terciária e Fallback Descentralizado (LibGen / Gateways IPFS):** Em caso de indisponibilidade das camadas primárias ou de títulos específicos, o sistema recorre ao ecossistema LibGen. Se os espelhos HTTP tradicionais responderem com bloqueios de borda (e.g., código `403 Forbidden` ou desafio Cloudflare), o script extrai o *hash* IPFS do documento e efetua o download através de múltiplos *gateways* IPFS públicos (e.g., `ipfs.io`, `dweb.link`, `cloudflare-ipfs.com`).

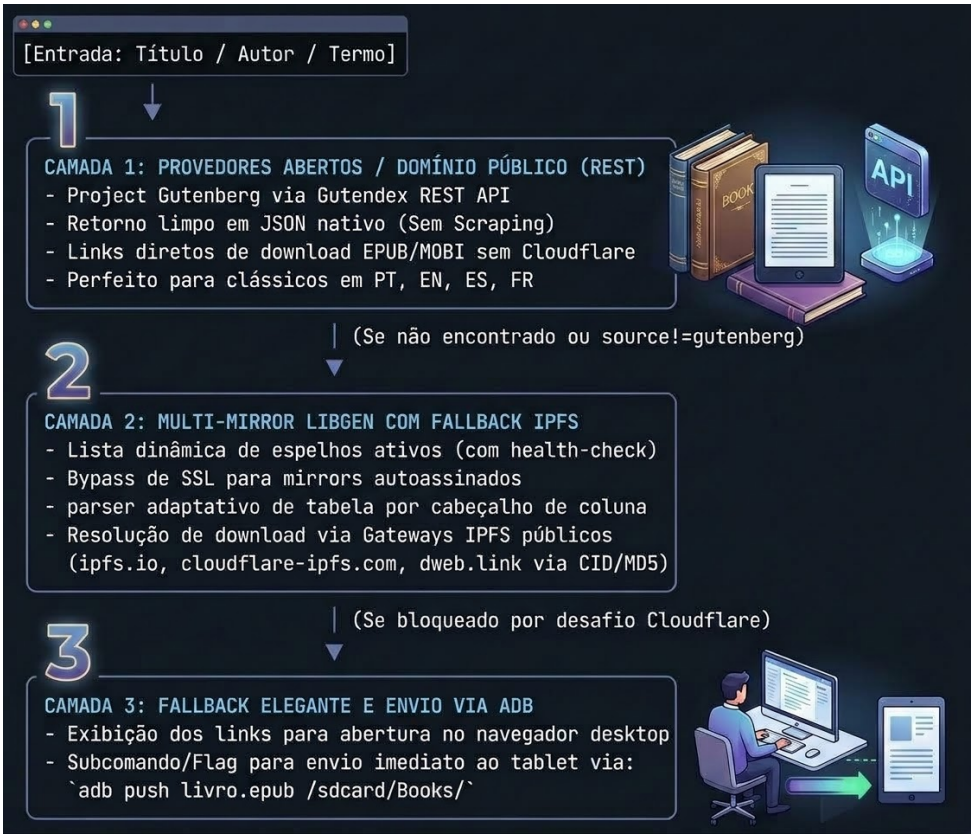


Figura 1 – Fluxo condicional de resiliência e *fallback* executado pelo utilitário `fetchbook`.

4.4.8.2 Integração Híbrida Bash-Python

Embora a estrutura principal do `fetchbook` permaneça em Bash para manter a compatibilidade com o ambiente de terminal Unix da ferramenta `ink`, a manipulação de dados complexos foi transferida para um interpretador Python *inline*.

O script executa blocos Python via `python3 -c` para realizar o *parsing* das estruturas JSON retornadas pelas APIs REST. Esta abordagem elimina a fragilidade de expressões regulares no tratamento de strings codificadas em UTF-8, sanitiza os títulos dos livros para evitar injeção de comandos no *shell* e gerencia a lógica de seleção de *gateways* IPFS quando uma falha HTTP é detectada.

4.4.8.3 Transferência Automatizada para o Dispositivo (`pushToDevice`)

Após a conclusão bem-sucedida do download e validação da integridade do arquivo `.epub` ou `.pdf`, o script aciona a função `pushToDevice`, que executa a verificação de estado do *daemon* ADB, mapeia o diretório de destino configurado no dispositivo (por padrão, `/sdcard/Books/` ou o diretório de biblioteca do KOReader), transfere o arquivo via `adb push <local_file_path> <remote_directory>` e emite um sinal de atualização do armazenamento de mídia (`android.intent.action.MEDIA_SCANNER_SCAN_FILE`) para que a nova obra seja indexada imediatamente pela biblioteca do leitor digital sem a

necessidade de reiniciar o aplicativo.

5 RESULTADOS E DISCUSSÃO

5.1 Consolidação da Primeira Versão Estável

Após sucessivos ciclos de testes e refatorações, o conjunto de ferramentas do InkDroid alcançou o estado de sua primeira versão estável (v1.0), consolidando-se como um utilitário funcional para a ressignificação de tablets Android legados. Inicialmente concebido como uma sequência linear e rígida de comandos imperativos em terminal, o projeto evoluiu para uma arquitetura modular estruturada em Shell Script (Bash) e Python, organizada em funções independentes e reutilizáveis.

Essa reestruturação viabilizou a criação do utilitário de linha de comando `ink`, que disponibiliza fluxos de execução interativos e não interativos através de parâmetros formais de linha de comando (*flags*). O script de instalação principal (`InkDroid_install.sh`) passou a permitir a execução seletiva de rotinas específicas, como `-only-debloat`, `-only-apps` ou `-only-display`, além da automação completa via confirmação automática com `-y`.

A estabilidade da ferramenta foi assegurada pela inclusão de mecanismos de validação preventiva de hardware e ambiente, e testes práticos, feitos no modelo proposto desde o início, o tablet Multilaser M8 4G. A rotina `checkDevice()` passou a exigir que apenas um dispositivo esteja conectado e devidamente autorizado via Depuração USB antes de disparar qualquer intervenção. Caso nenhum aparelho ou múltiplos terminais sejam detectados, a execução é interrompida com códigos de erro claros, prevenindo a aplicação acidental de comandos em terminais incorretos. Complementarmente, a integração de mecanismos de *fallback* como o processamento de listas em Python quando o utilitário `jq` não se encontra disponível no sistema hospedeiro conferiu robustez à execução em diferentes distribuições Linux.

Entretanto, o Shell Script em Bash não executa nativamente no sistema Windows, o que restringe a ferramenta a ambientes macOS e Linux, os quais seguem as diretrizes e padrões POSIX. Usuários de Windows necessitam recorrer a opções de compatibilidade como o WSL (*Windows Subsystem for Linux*).

5.2 Objetivos Alcançados

Tomando como base os objetivos definidos na pesquisa e a implementação técnica realizada no tablet Multilaser M8 4G, a solução cumpriu com êxito as metas projetadas para os níveis de aplicação e de *framework*:

1. **Desoneração Sistêmica via Debloat Automatizado (Nível de Aplicação):** A automação implementou a remoção declarativa e não invasiva de pacotes catalogados no arquivo `blacklist.json`, mitigando os efeitos do envelhecimento de software (*software aging*). Através da instrução `pm uninstall -k -user 0`, a ferramenta desinstala redes sociais, jogos pré-instalados e os serviços intrusivos do *Google Mobile Services* (GMS) exclusivamente no espaço do usuário primário. Essa limpeza eliminou telemetrias de fundo que drenavam ciclos de processamento e memória RAM, registrando todo o histórico de execuções com carimbo temporal no arquivo de auditoria.

2. **Transformação em Ambiente de Leitura e Fluxo de Uso:** O sistema automatizou a transferência e instalação dos aplicativos dedicados à leitura (KOReader) e substituiu o gerenciador de tela inicial padrão pelo Olauncher por meio da API RoleManager do Android. Essa troca de interface estabeleceu um ambiente minimalista baseado estritamente em tipografia, eliminando notificações e distrações visuais de acordo com os preceitos do Minimalismo Digital de Newport (2019). O ecossistema foi complementado pela rotina `ink push`, capaz de transferir e-books diretamente para a pasta `/sdcard/Books/` do aparelho e acionar via transmissão de sinal *broadcast* nativo para indexação imediata no leitor.

3. **Ergonomia Visual e Simulação de E-ink:** Para conter os fatores associados à Síndrome da Visão do Computador (CVS) decorrentes do uso de telas LCD retroiluminadas, o InkDroid injetou parâmetros diretamente no banco relacional `settings.db` e no gestor de janelas. A ferramenta desativou completamente a escala de animações do sistema, diminuindo consideravelmente travamentos gráficos da GPU legada. O script configurou a taxa de atualização para a menor frequência suportada pelo painel (30 Hz) e estruturou perfis ergonômicos acionáveis via terminal: a conversão monocromática em escala de cinza através do daltonizador de acessibilidade e a atenuação de luz azul noturna.

4. **Preservação Energética e Contenção de Processos:** Em conformidade com a literatura técnica que aponta os processos em segundo plano e a conectividade como os maiores causadores de consumo em repouso, a automação ativou a destruição imediata de atividades em segundo plano. O perfil de economia foi fixado em nível máximo, o tempo de bloqueio de tela foi ampliado para 30 minutos a fim de evitar cortes durante leituras densas, e varreduras ocultas de Wi-Fi, Bluetooth BLE, motores de vibração háptica e atualizações remotas OTA foram completamente suprimidas.

5. **Arquitetura de Reversibilidade:** A conformidade com a segurança do usuário foi garantida pelo utilitário, que lê o arquivo de configurações para reinstalar os pacotes desativados através do comando `cmd package install-existing`, reverte todas as

configurações de framework, e retorna o launcher padrão restaurando o tablet ao estado de fábrica sem necessidade de formatação de partições nem perda de arquivos pessoais.

5.3 Limitações e Desafios de Intervenção em Nível de Kernel

Apesar da previsão metodológica inicial de intervir sobre a frequência dos núcleos do processador e aplicar governadores de energia específicos no nível de Kernel, essas alterações não foram executadas nesta primeira versão estável da ferramenta. Essa decisão técnica fundamentou-se em restrições arquiteturais intrínsecas ao ecossistema Android e no modelo de segurança adotado pelo projeto:

Para alterar parâmetros de hardware em tempo de execução no Linux, tais como a escrita nos arquivos virtuais do subsistema sysfs em `/sys/devices/system/cpu/cpu*/cpufreq/`, é imperativo possuir acesso de root ou desbloquear o *bootloader* para gravar uma imagem customizada de Kernel. Como o InkDroid estabeleceu o compromisso de operar de maneira estritamente não invasiva, baseando-se unicamente nas permissões nativas concedidas pelo usuário `shell` da interface ADB, a manipulação direta do escalonador de frequência do processador é bloqueada pelas políticas de segurança do SELinux e pelas permissões do sistema.

Outro desafio que precisaria ser superado é que, em sistemas móveis, as rotinas de gerenciamento de frequência e voltagem são profundamente dependentes da arquitetura do chip (*System-on-Chip*). Diferentes fabricantes (como MediaTek, Exynos e Qualcomm) implementam controladores de clock, faixas térmicas e nós de arquivos proprietários. A tentativa de automatizar rotinas de *underclock* por meio de scripts genéricos em nível de Kernel resultaria em elevado risco de travamento crítico (*kernel panic*), falhas de inicialização (*bootloops*) ou até a inutilização permanente do hardware (*hard brick*).

Existem formas de contornar esses desafios, porém a complexidade de automação e o tempo de pesquisa também se tornam impeditivos. Tornar a intervenção de Kernel automática, segura e agnóstica ao fabricante exigiria o desenvolvimento e a compilação de módulos de Kernel sob medida para a placa-mãe do Multilaser M8 4G. Esse nível de engenharia reversa de baixo nível demandaria um ciclo de pesquisa extenso, o que comprometeria a portabilidade, a reprodutibilidade e a estabilidade imediata exigidas para a entrega da solução no tempo do projeto.

Dessa forma, optou-se por concentrar os esforços de conservação de energia no nível de Framework, forçando a suspensão agressiva de processos, limitando a taxa de atualização gráfica e impondo o modo econômico permanente, estratégia que demonstrou alta estabilidade e dispensou a violação da segurança do sistema.

O que também pode ser implementado é o uso do módulo de customização de *kernel* para Android denominado KernelSU. Apesar de exigir root, ele concede ao usuário a possibilidade de efetuar *undervolt*, *underclock* etc. Entretanto, a ferramenta apenas poderia automatizar a sua instalação caso o dispositivo já contasse com acesso privilegiado previamente concedido.

6 CONSIDERAÇÕES FINAIS

O desenvolvimento do InkDroid comprovou a viabilidade técnica de reutilizar dispositivos móveis considerados obsoletos através de intervenções conscientes de engenharia de software. Ao atuar sobre o tablet Multilaser M8 4G da rede pública estadual da Bahia, o projeto demonstrou que aparelhos descartados ou retidos na "hibernação doméstica" não apresentam incapacidade física real, mas sim uma sobrecarga artificial imposta pelo acúmulo de softwares mal otimizados, serviços de telemetria e interfaces projetadas para capturar compulsivamente a atenção.

Os objetivos delineados na pesquisa foram amplamente cumpridos no âmbito das camadas de aplicação e *framework*. A criação de uma interface de linha de comando robusta, orientada a funções e modular permitiu a realização de um *debloat* radical e seguro, a implantação de uma suíte de leitura focada e o ajuste de parâmetros ergonômicos que amenizam os desconfortos visuais característicos da Síndrome da Visão do Computador. Ao mesmo tempo, a pesquisa mapeou as fronteiras e limitações operacionais da plataforma Android, evidenciando as barreiras de privilégios que tornam a automação em nível de Kernel inviável sem concessões ao modelo de segurança e à portabilidade da ferramenta.

Como contribuição prática e social, o InkDroid consolida uma prova de conceito alinhada às práticas de Tecnologia da Informação Verde (Green IT) e à economia circular. Ao estender o ciclo de vida útil de hardwares educacionais legados e convertê-los em leitores de livros digitais funcionais, a solução oferece uma alternativa contra o descarte prematuro e a geração de resíduos eletroeletrônicos (*e-waste*), democratizando o acesso a ferramentas de estudo e promovendo o trabalho focado (*Deep Work*) sem custos adicionais aos estudantes e à sociedade.

Para trabalhos futuros, planeja-se:

1. Aprofundar a investigação das intervenções em nível de Kernel em ambientes controlados com acesso root, analisando a viabilidade de compilação de kernels mínimos com governadores estáticos de baixa frequência;

2. Realizar ensaios experimentais quantitativos de consumo energético utilizando analisadores físicos de corrente ou ferramentas como BatteryStats, mensurando em Joules a autonomia ganha antes e depois do processo de debloat;

3. Conduzir testes com usuários aplicando metodologias de avaliação ergonômica e taxas de piscar de olhos para quantificar o conforto visual obtido pela simulação monocromática em comparação ao painel colorido padrão.

Referências Bibliográficas

ABREE. *Relatório de Resultados de Logística Reversa de Eletroeletrônicos e suas Embalagens 2023-2024*. São Paulo, 2024. 9

ANDROID DEVELOPERS. *Android Debug Bridge (adb)*. 2025. Disponível em: <https://developer.android.com/tools/adb>. Acesso em: 1 ago. 2026. 10, 12, 15, 16, 17, 18

AOSP. *System Architecture and Platform Documentation*. 2024. Disponível em: <https://source.android.com>. Acesso em: 15 mar. 2026. 15, 16, 17

BALDé, C. P. et al. *The Global E-waste Monitor 2024: Quantities, flows and the global e-waste challenge*. Geneva/Bonn, 2024. 7, 9

BENEDETTO, S. et al. E-readers and visual fatigue: Comparison between e-ink and lcd. *PLOS ONE*, v. 8, n. 12, p. e83676, 2013. 10, 17, 18

KNEZ, I. Effects of colour of light on non-visual psychological processes. *Journal of Environmental Psychology*, v. 21, n. 2, p. 201–208, 2001. 18

MONTEIRO, E. et al. Analysis of energy consumption on android devices for developers: A systematic mapping study. *SBC Reviews on Computer Science*, v. 29, n. 1, 2023. 8, 14

NEWPORT, C. *Minimalismo Digital: para uma vida profunda em um mundo superficial*. Rio de Janeiro: Alta Books, 2019. 9, 14, 21, 27

PARNAS, D. L. Software aging. In: *Proceedings of the 16th International Conference on Software Engineering*. Hamilton: IEEE, 1994. p. 279–287. 7, 13

PONTES, J. C. N. *Análise de desempenho e consumo de energia de parâmetros TCP/IP em dispositivos móveis*. 71 p. Dissertação (Mestrado) — Centro de Informática, Universidade Federal de Pernambuco, Recife, 2017. 11

Sá, E. C. *Síndrome da visão do computador e função visual em trabalhadores usuários de computador de um hospital público universitário de São Paulo: prevalência e fatores associados*. 107 p. Tese (Doutorado) — Faculdade de Saúde Pública, Universidade de São Paulo, São Paulo, 2016. 7, 10, 17, 18

TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016. 12, 15, 16

YAGHMOUR, K. *Embedded Android: Porting, Extending, and Customizing*. 1. ed. Sebastopol: O'Reilly Media, 2013. 17

GLOSSÁRIO

ADB (*Android Debug Bridge*)
Ferramenta de linha de comando oficial do AOSP que opera em modelo cliente-servidor, permitindo a comunicação, envio de instruções e automação de *scripts* entre a estação de trabalho e o dispositivo Android.

AOSP (*Android Open Source Project*)
Projeto de código aberto mantido pela Google que disponibiliza a estrutura base e a arquitetura do sistema operacional Android.

API (*Application Programming Interface*)
Interface de Programação de Aplicações; conjunto de rotinas e padrões de software que permite a comunicação entre diferentes componentes de sistema.

ART (*Android Runtime*)
Ambiente de execução do Android responsável por traduzir o código das aplicações em instruções de máquina executadas pelo processador.

Blacklist Arquivo de configuração estruturado (em formato JSON) utilizado para catalogar e especificar os identificadores de pacotes (*package names*) alvos de remoção.

Bloatware Softwares e aplicativos pré-instalados por fabricantes ou operadoras que operam em segundo plano e consomem memória e ciclos de processamento desnecessários.

CLI (*Command Line Interface*)
Interface de linha de comando utilizada para a execução de programas e *scripts* em modo texto no terminal.

CPU (*Central Processing Unit*)
Unidade Central de Processamento; componente de hardware responsável pela execução dos cálculos e instruções do sistema.

CVS (*Computer Vision Syndrome*)
Síndrome da Visão do Computador; condição ocupacional caracterizada por fadiga e desconforto ocular resultantes do uso prolongado de telas e terminais de vídeo.

Daemon Processo do sistema operacional que executa de forma contínua em segundo plano sem a necessidade de interatividade direta com o usuário (ex.: `adbd`).

Debloat Processo sistemático de limpeza e desinstalação de pacotes, aplicativos e serviços redundantes de um sistema operacional.

Deep Work Trabalho profundo; conceito que se refere ao estado de concentração contínua e redução de estímulos/distrações induzido por dispositivos de propósito único.

Desejabilidade Composta
Modelo de análise estatística e multicritério utilizado para calcular o ponto de equilíbrio ótimo entre métricas de desempenho (*throughput*) e consumo de energia.

Dispositivo Legado (*Legacy Device*)
Hardware perfeitamente funcional que se tornou obsoleto em relação às exigências de softwares e atualizações recentes.

DPI (*Dots Per Inch*)
Densidade de pontos por polegada; métrica utilizada para ajustar a escala e a proporção dos elementos visuais da interface do Android.

E-ink / E-reader
Tecnologia de tela de tinta eletrônica reflexiva e dispositivos leitores de livros digitais projetados para alta ergonomia visual.

E-waste Resíduos de equipamentos eletroeletrônicos acumulados devido ao descarte e à obsolescência de hardware.

GMS (*Google Mobile Services*)
Conjunto de aplicativos, telemetrias e serviços proprietários da Google que executam em segundo plano no sistema Android.

HAL (*Hardware Abstraction Layer*)
Camada de Abstração de Hardware que faz a ponte entre o *framework* do Android e os *drivers* do Kernel Linux.

IPC (*Inter-Process Communication*)
Comunicação entre processos; mecanismo que permite a troca segura de dados e chamadas entre diferentes processos no sistema (como a arquitetura Binder).

JSON (*JavaScript Object Notation*)
: Formato leve e estruturado para troca e armazenamento de dados em pares de chave-valor.

Kernel
: Núcleo do sistema operacional (baseado em Linux no Android) responsável pela alocação de memória, escalonamento e comunicação com o hardware.

Launcher
: Aplicativo gerenciador da tela de início e da organização de ícones e interfaces gráficas do Android.

Minimalismo Digital
: Filosofia de otimização do uso de ferramentas digitais para reduzir a fragmentação da atenção e focar em tarefas prioritárias.

RAM (*Random Access Memory*)
: Memória de acesso aleatório onde são armazenados os dados temporários dos processos em execução.

Settings.db
: Banco de dados interno do Android responsável por armazenar as configurações globais e comportamentos de sistema.

Shell
: Interpretador de comandos do sistema operacional Unix/Linux responsável por receber e executar comandos de terminal.

Slowdown
: Métrica de degradação de desempenho que avalia o atraso relativo de um processo sob condições de carga do sistema.

Software Aging
: Envelhecimento de software; conceito que explica a perda progressiva de fluidez em sistemas motivada pelo acúmulo de serviços e atualizações pesadas em hardwares antigos.

SurfaceFlinger
: Serviço do Android responsável por compor as superfícies visuais e controlar a taxa de atualização enviada ao display.

UI (*User Interface*)
: Interface do usuário; conjunto de componentes visuais através dos quais o usuário interage com o sistema.

Underclocking
: Redução proposital da frequência de operação do processador com o objetivo de diminuir o consumo de bateria e a dissipação térmica.

ALSA (*Advanced Linux Sound Architecture*)
: Arquitetura de som e conjunto de drivers do Kernel Linux responsável por gerenciar e se comunicar diretamente com os controladores físicos de áudio do dispositivo.

AudioFlinger Serviço interno do ecossistema de áudio do Android responsável pela mixagem digital, consolidando múltiplos fluxos sonoros concorrentes em uma única faixa PCM antes do envio à HAL.

AudioServer Daemon nativo escrito em C++ que executa no Daemon Space, responsável por processar e gerenciar requisições e políticas de áudio do sistema.

AudioService Módulo executado no User Space que gerencia configurações de alto nível do usuário, tais como volume, modo silencioso e permissões de foco de reprodução (*Audio Focus*).

Binder Barramento de comunicação interprocessos (*IPC*) nativo do Android que empacota mensagens e valida privilégios de segurança e isolamento de processos (*sandboxing*) entre o Daemon Space e o User Space.

Broadcast Mecanismo de transmissão assíncrona de sinais do Android utilizado pela automação para notificar subsistemas do sistema operacional — como o disparo do *MEDIA_SCANNER* para indexar novos e-books sem reiniciar o aplicativo leitor.

DAC (*Digital to Analog Converter*)
Conversor Digital-Analógico; circuito eletrônico encarregado de transformar o sinal digital modulado (PCM) em variações de sinal elétrico analógico enviadas ao alto-falante.

Doze Mode Mecanismo nativo de economia energética do Android que restringe o consumo de bateria e suspende atividades e varreduras de rede quando a tela permanece desligada.

Green IT (*Tecnologia da Informação Verde*)
Abordagem da engenharia que promove a sustentabilidade digital, o reuso de equipamentos legados e a redução da pegada ecológica e do lixo eletrônico.

Hibernação Doméstica
Fenômeno em que aparelhos eletrônicos funcionais retêm-se acumulados nas residências após a perda de utilidade primária, sem serem descartados ou reciclados adequadamente.

IPFS (*InterPlanetary File System*)
Protocolo descentralizado de armazenamento e compartilhamento ponto a ponto (P2P), utilizado pela ferramenta de busca para contornar bloqueios e obter livros por meio de *gateways* públicos.

JNI (*Java Native Interface*)
: Mecanismo de ponte de programação que permite que rotinas em linguagens de alto nível (Java/Kotlin) façam chamadas a bibliotecas e daemons compilados em C/C++.

Juice Jacking
: Vetor de ataque cibernético em que dispositivos móveis são comprometidos ou têm dados extraídos ao serem plugados em portas de carregamento USB públicas desprotegidas.

LCD (*Liquid Crystal Display*)
: Tecnologia de tela transmissiva retroiluminada com lâmpadas ou LEDs que projeta luz contínua diretamente nos olhos do usuário, apresentando maiores índices de fadiga visual em comparação ao papel ou ao *E-ink*.

OTA (*Over-The-Air*)
: Mecanismo de atualização remota e sem fio de pacotes e do sistema operacional distribuído pelos fabricantes.

PCM (*Pulse Code Modulation*)
: Modulação por Código de Pulsos; formato digital não compactado que armazena as amostras sonoras decodificadas antes da conversão para sinais elétricos.

Repurposing (*Ressignificação de Hardware*)
: Prática de engenharia de software voltada a adaptar dispositivos móveis considerados obsoletos para funções especializadas e dedicadas (como um *E-reader*).

REST (*Representational State Transfer*)
: Padrão de arquitetura de comunicação em rede utilizado pelo utilitário do projeto para requisitar metadados de obras em formato JSON em APIs abertas (como a do Project Gutenberg).

RSA
: Algoritmo de criptografia assimétrica baseado em chaves pública e privada utilizado pelo protocolo ADB para validar a autorização de depuração entre a máquina hospedeira e o tablet.

Sandbox
: Mecanismo de segurança do Android baseado no Linux que isola os processos de cada aplicativo em um ambiente restrito e independente.

Scraping (*Web Scraping*)
: Técnica de extração automatizada de dados e raspagem de páginas web em HTML, substituída no projeto por APIs estruturadas e redes distribuídas para evitar desafios de bloqueio.

Stagefright / Codec2
: Subsistema multimídia do Android responsável pela decodificação e processamento de formatos de mídia comprimidos.

Throughput
: Métrica quantitativa de desempenho que afere a taxa de vazão ou o volume efetivo de tarefas processadas pelo sistema por unidade de tempo.

UID (*User Identifier*)
: Identificador de usuário exclusivo atribuído pelo sistema a cada aplicativo instalado, garantindo o controle rígido de privilégios de acesso.

Window Manager (*WM*)
: Serviço de gerenciamento de exibição do sistema que controla a disposição das janelas e a densidade de renderização da tela (*wm density*).

APÊNDICE A – Acesso ao Repositório de Código Aberto

Este apêndice disponibiliza o acesso público, as diretrizes de governança e a estrutura organizacional do repositório de código-fonte aberto (*open-source*) desenvolvido no âmbito deste Trabalho de Conclusão de Curso.

Identificação e Acesso ao Repositório

O ecossistema de scripts de automação, ferramentas de linha de comando (CLI) e arquivos de configuração que compõem o **InkDroid** encontram-se hospedados na plataforma GitHub:

- **Título do Projeto:** InkDroid: Otimização e Ressignificação de Tablets Android Legados via ADB
- **Autor e Mantenedor:** Alisson Gabriel dos Santos Alves
- **URL Oficial de Acesso:** <https://github.com/bagi-sh/InkDroid>
- **Ramo Padrão (*Default Branch*):** `main`
- **Licença de Software:** MIT

Instruções de Clonagem e Execução

Para fins de replicação e auditoria dos experimentos descritos nesta pesquisa, o repositório pode ser clonado em qualquer distribuição Linux por meio do utilitário Git:

```
# 1. Clonagem do repositório oficial
git clone https://github.com/bagi-sh/InkDroid.git

# 2. Navegação até o diretório do projeto
cd InkDroid

# 3. Concessão de permissões de execução
chmod +x ./*
```

```
# 4. Execução interativa da automação
./ink setup
```